# Software Requirements Specification

Writer's Creative Ideas Information System

Academic / Engineering Project

Draft for engineering baseline review
Version 2.0

| | |
|---|---|
| Document status | Requirements baseline draft |
| Methodology | Socratic Knowledge Elicitation & Documentation (SKED) |
| Guideline | IEEE 830-style SRS structure |
| Target product | Offline-first mobile client with REST API backend |

# Contents


**1 Introduction** **3**

1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Intended Audience and Reading Guidance . . . 3
1.4 Definitions . . . 3
1.5 References . . . 4
1.6 Document Conventions . . . 4

**2 Overall Description** **4**

2.1 Product Perspective . . . 4
2.2 Product Functions . . . 4
2.3 User Classes and Characteristics . . . 4
2.4 Operating Environment . . . 5
2.5 Design and Implementation Constraints . . . 5
2.6 User Documentation . . . 5
2.7 Assumptions and Dependencies . . . 5

**3 Specific Requirements** **5**

3.1 External Interface Requirements . . . 5
3.1.1 User Interface . . . 5
3.1.2 Software Interfaces . . . 6
3.1.3 Communications Interfaces . . . 6
3.1.4 Data and Export Interface . . . 6
3.2 Functional Requirements . . . 6
3.2.1 Quick Capture and Entry Management . . . 6
3.2.2 Books, Chapters, and Associations . . . 7
3.2.3 Entry Linking . . . 7
3.2.4 Retrieval and Search . . . 7
3.2.5 Export . . . 8
3.2.6 Backend API and Synchronization . . . 8
3.3 Non-Functional Requirements . . . 9

**4 Acceptance Criteria** **10**

4.1 Functional Acceptance Criteria . . . 10
4.2 Non-Functional Acceptance Criteria . . . 12

**5 Behavioral Scenarios (Given–When–Then)** **13**

**6 Traceability and Outstanding Specification Details** **16**

# 1 Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) defines the agreed scope, externally observable behavior, quality attributes, acceptance criteria, and behavioral scenarios for the Writer's Creative Ideas Information System. It is intended for the project owner, analysts, designers, implementers, testers, and academic reviewers. Requirement identifiers are stable references for specifications, tickets, tests, and implementation traceability.

## 1.2 Scope

The product is an offline-first smartphone client backed by a server application and REST API. Writers can capture, organize, link, search, and export literary creative material. A user can manage Books and Chapters, create Entries with preset Entry Types and Tags, associate Entries with Books and Chapters, and link Entries to one another. The client persists a local working copy in SQLite, supports core workflows without a network connection, and synchronizes changes with the server when connectivity is available.

The MVP client is limited to iOS and Android. A web client, desktop client, manuscript editor, multi-user collaboration, publishing workflow, and complex project/deadline management are out of scope. Server-backed data access and synchronization are in scope; account and authentication behavior is not specified in this revision. Text export to device-local storage is included.

## 1.3 Intended Audience and Reading Guidance

Section 2 describes context, users, platform, and constraints. Section 3 contains the normative functional and non-functional requirements. Section 4 defines acceptance evidence for every requirement. Section 5 specifies representative Given–When–Then (GWT) behavioral scenarios. Requirements use the word "shall" to indicate a mandatory product obligation.

## 1.4 Definitions

**Book** A user-created container for creative material belonging to a literary project.
**Chapter** A named subdivision of a Book. Entries may be associated with a Chapter and with the Chapter's parent Book.
**Entry** A piece of creative material managed by the system. "Note" is a user-facing synonym, not a separate data entity.
**Entry Type** The preset primary classification of an Entry: Character, Scene, Plot Idea, Worldbuilding, Dialogue/Phrase, or General Note.
**Tag** A user-applied flexible label that supplements, and does not replace, an Entry Type.
**Standalone Entry** An Entry with no current Book or Chapter associations.
**Book Association** A direct relationship between an Entry and a Book as a whole.
**Chapter Association** A relationship between an Entry and a Chapter. It automatically implies association with the Chapter's parent Book. A direct Book Association is tracked separately.
**Entry Link** An unlabeled, symmetric relationship between two Entries, distinct from Book and Chapter associations.
**Quick Capture** The workflow for creating an Entry with minimal required input. A General Note can be created with Content only.
**Display Title** The title shown for an Entry. It is the explicit Title when nonblank; otherwise, it is derived from Content under REQ-F-003.
**Book Project Export** A text export of a Book and the Entries associated with it directly or through its Chapters.

**Pending Change** A locally persisted create, update, or delete that has not yet been acknowledged by the server.
**Sync Cursor** An opaque server-issued position used by a client to request changes made since its previous synchronization.
**Sync Conflict** A server-side version conflict in which the same record was changed locally and remotely since the client's last synchronization.

### 1.5 References

- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*; used as a structural guideline. This document is not a claim of formal IEEE compliance or certification.
- Approved SKED decisions recorded in the project conversation through 2026-09-26.

### 1.6 Document Conventions

Identifiers beginning `REQ-F-` denote functional requirements. Identifiers beginning `REQ-NF-` denote non-functional requirements. Acceptance criteria use corresponding `AC-F-` and `AC-NF-` identifiers. GWT scenarios complement the individual criteria and do not replace them.

## 2 Overall Description

### 2.1 Product Perspective

The product uses a mobile client and a server application. The client maintains a durable local SQLite working copy via Expo SQLite, communicates with the server through a versioned REST API, and synchronizes pending local changes and server changes when connected. The mobile operating system provides the runtime, accessibility services, and local file handling used for text export. While offline, the client continues to use its local data and queues changes for later synchronization.

### 2.2 Product Functions

At a high level, the product provides:

- quick capture and lifecycle management of Entries;
- preset Entry Types and user-defined Tags;
- Book and Chapter organization, including many-to-many Entry-to-Book association;
- automatic parent-Book association for Chapter-associated Entries;
- symmetric links between Entries;
- case-insensitive full-text search and combined search/type/tag filtering;
- local text export of an Entry or Book project;
- synchronization of local changes to the server and retrieval of remote changes; and
- internal local diagnostic logging without a dedicated user-facing log interface.

### 2.3 User Classes and Characteristics

The primary user class is an individual writer, novelist, screenwriter, or storyteller who needs to capture ideas on the move and later organize and retrieve a large volume of heterogeneous creative material. The MVP has one local user context per device; account identity, authentication, and multi-user behavior are not defined.

### 2.4 Operating Environment

- Mobile application developed with React Native, Expo, and TypeScript.
- Android 10.0 or later (API level 29 or later).
- iOS 15.0 or later.
- Minimum supported logical viewport: 360 by 640 device-independent pixels (dp).
- On-device SQLite persistence using `expo-sqlite`, with a remote server reachable over HTTPS when connected.
- Core application workflows must function without network connectivity; synchronization requires network connectivity.

### 2.5 Design and Implementation Constraints

- The MVP client is mobile-only; web and desktop clients are excluded.
- The client shall retain a local working copy; the server is the shared synchronization endpoint when the network is available.
- The REST API transport and payload format shall follow the communications requirements in Section 3.
- Authentication, account lifecycle, and multi-user access control are not specified by this revision.
- The Entry Types are a fixed preset set. Flexible categorization is provided by Tags, not user-created Entry Types.
- Book and Chapter deletion must not cascade into deletion of Entries.
- Entry Links are unlabeled and symmetric.
- The codebase shall use TDD and meet the coverage thresholds in REQ-NF-014.

### 2.6 User Documentation

In-app help content, onboarding, and end-user documentation are not specified by the approved MVP decisions. Their inclusion and format remain for project planning.

### 2.7 Assumptions and Dependencies

- The device has functional local storage and an operating-system-supported means of saving or selecting a local destination for exported files.
- Availability of a file destination or permission may vary by platform; export failure must be reported as specified.
- Normal operating conditions for performance measurement, export text layout, diagnostic-log retention, and some detailed matching behavior require a verification/design decision; these are recorded in Section 6 rather than silently assumed.

## 3 Specific Requirements

### 3.1 External Interface Requirements

#### 3.1.1 User Interface

The interface shall be designed for small smartphone screens and quick on-the-go capture. It shall provide workflows to create and manage Entries, Books, Chapters, Tags, and Entry Links; retrieve Entries; and initiate exports. Detailed screen layouts, navigation structure, and visual design are outside this SRS.

### 3.1.2 Software Interfaces

The mobile client shall use React Native, Expo, TypeScript, and `expo-sqlite` for local relational persistence. It shall use platform capabilities as needed for local text-file export and accessibility. The client shall communicate with the backend REST API described below.

### 3.1.3 Communications Interfaces

Client-server communication shall use HTTPS. API requests and responses shall use UTF-8 encoded JSON and RESTful resource semantics. The versioned API base path shall be `/api/v1`. Resource endpoints shall include `/entries`, `/books`, `/chapters`, `/tags`, `/entry-tag-associations`, `/entry-book-associations`, `/entry-chapter-associations`, and `/entry-links`; standard HTTP methods shall be used to create, retrieve, update, and delete resources. Synchronization shall use `POST /api/v1/sync/push` for client changes and `GET /api/v1/sync/pull?cursor={cursor}` for server changes. Requests shall be independently processable and responses shall use HTTP status codes to report success, validation errors, and version conflicts. The client shall not require network access for core capture, organization, search, or export workflows.

### 3.1.4 Data and Export Interface

The local database shall represent Entries, Books, Chapters, Tags, Book Associations, Chapter Associations, Entry Links, pending changes, synchronization metadata, and diagnostic events. API resource representations shall identify the resource and its relationships and carry version information needed to detect concurrent changes. Synchronization responses shall identify acknowledged changes, remote changes, deletions, and the next Sync Cursor. Export output shall be a `.txt` file saved to device-local storage. UTF-8 encoding is required. Exact export metadata, punctuation, and file-naming conventions remain unspecified.

## 3.2 Functional Requirements

### 3.2.1 Quick Capture and Entry Management

| ID | Requirement |
|---|---|
| REQ-F-001 | The system shall allow the user to create an Entry through a Quick Capture workflow. |
| REQ-F-002 | The system shall require nonblank Content to create an Entry. Title shall be optional. |
| REQ-F-003 | When Title is blank, the system shall derive the Display Title from Content. The 25-character limit applies only to the body content of the Display Title; an ellipsis ("...") is additional. If truncation occurs because Content exceeds 25 characters or because a newline occurs within the first 25 characters, the system shall append "..." after the retained body content. Content after the first newline shall not appear in the Display Title. |
| REQ-F-004 | The system shall provide exactly these preset Entry Types: Character, Scene, Plot Idea, World-building, Dialogue/Phrase, and General Note. |
| REQ-F-005 | The system shall allow the user to create a General Note with Content as the only required input. |
| REQ-F-006 | The system shall allow the user to view an Entry's Content, explicit Title or generated Display Title, Entry Type, Tags, Book Associations, Chapter Associations, and Entry Links. |

| ID | Requirement |
|---|---|
| REQ-F-007 | The system shall allow the user to edit an Entry's Content and Title, change its Entry Type, and add or remove its Tags, Book Associations, and Chapter Associations. |
| REQ-F-008 | The system shall allow the user to delete an Entry. Deleting an Entry shall not leave active Entry Links to it. |
| REQ-F-009 | The system shall allow the user to apply Tags to Entries and remove Tags from Entries. |
| REQ-F-010 | The system shall allow an Entry to have no Book or Chapter Associations and remain available as a Standalone Entry for later association. |

### 3.2.2 Books, Chapters, and Associations

| ID | Requirement |
|---|---|
| REQ-F-011 | The system shall allow the user to create, view, edit, and delete Books. |
| REQ-F-012 | The system shall allow the user to create, view, edit, and delete Chapters within a Book. |
| REQ-F-013 | The system shall allow the user to directly associate an Entry with a Book as a whole and remove that Book Association. |
| REQ-F-014 | The system shall allow an Entry to have Book Associations with multiple Books. |
| REQ-F-015 | The system shall allow the user to associate an Entry with a Chapter within a Book and remove that Chapter Association. |
| REQ-F-016 | Associating an Entry with a Chapter shall automatically associate the Entry with that Chapter's parent Book. The system shall distinguish this Chapter-derived association from a separately created direct Book Association. |
| REQ-F-017 | Deleting a Book or Chapter shall not delete associated Entries. The system shall remove associations to the deleted Book or Chapter while preserving associations to other extant Books or Chapters. An Entry with no remaining associations shall remain available as a Standalone Entry. |
| REQ-F-018 | Removing or deleting a Chapter Association shall preserve a separately created direct Book Association to the Chapter's parent Book. A Book association that exists only because of that Chapter Association shall not remain solely by virtue of the deleted Chapter Association. |

### 3.2.3 Entry Linking

| ID | Requirement |
|---|---|
| REQ-F-019 | The system shall allow the user to link two Entries without assigning a label to the Entry Link. |
| REQ-F-020 | Entry Links shall be symmetric: linking Entry A to Entry B shall make the link visible from both Entries. |
| REQ-F-021 | The system shall allow the user to remove an Entry Link. Removing the link shall not delete either Entry. |

### 3.2.4 Retrieval and Search

| ID | Requirement |
|---|---|
| REQ-F-022 | The system shall provide case-insensitive full-text search across both Entry Title and Entry Content and display matching Entries. A generated Display Title shall be searchable as the Entry's displayed title. |
| REQ-F-023 | The system shall allow the user to filter Entries by Entry Type. When multiple Entry Types are selected, an Entry shall match all selected Entry Types (strict AND logic). |
| REQ-F-024 | The system shall allow the user to filter Entries by Tag. When multiple Tags are selected, an Entry shall match all selected Tags (strict AND logic). |
| REQ-F-025 | When Search, Entry Type, and Tag criteria are combined, the system shall use strict AND logic across categories: a result shall satisfy every active criterion. Strict AND logic shall also apply among multiple selected Entry Types and among multiple selected Tags. |

### 3.2.5 Export

| ID | Requirement |
|---|---|
| REQ-F-026 | The system shall allow the user to export an Entry as a `.txt` file to device-local storage. The exported file shall contain the Entry's Content. |
| REQ-F-027 | The system shall allow the user to export a Book project as a `.txt` file to device-local storage. |
| REQ-F-028 | A Book Project Export shall include Entries associated directly with the Book and Entries associated with any of its Chapters. Chapter-associated Entries shall appear under their respective Chapter headings. Chapters shall be ordered by an internal integer position field in ascending order when that field is available; otherwise, they shall be ordered by `created_at` ascending. Direct Book-level Entries not placed under a Chapter shall appear in a dedicated "General Book Notes" section at the end. |
| REQ-F-029 | Each Entry shall appear no more than once in a Book Project Export. An Entry associated with a Chapter and its parent Book shall appear under the Chapter heading and not be duplicated in "General Book Notes". |

### 3.2.6 Backend API and Synchronization

| ID | Requirement |
|---|---|
| REQ-F-030 | The backend API shall allow the client to create, retrieve, update, and delete Entries through the `/api/v1/entries` resource. Successful changes shall be represented in JSON and assigned a server version. |
| REQ-F-031 | The backend API shall allow the client to create, retrieve, update, and delete Books through the `/api/v1/books` resource. |
| REQ-F-032 | The backend API shall allow the client to create, retrieve, update, and delete Chapters through the `/api/v1/chapters` resource, including their parent-Book relationship. |
| REQ-F-033 | The backend API shall allow the client to create, retrieve, update, and delete Tags through the `/api/v1/tags` resource and to represent Entry-to-Tag assignments. |

| ID | Requirement |
|---|---|
| REQ-F-034 | The backend API shall allow the client to create, retrieve, and delete Entry-to-Tag Associations, Entry-to-Book Associations, Entry-to-Chapter Associations, and Entry Links through their respective resources. It shall preserve the relationship and deletion behavior in REQ-F-009, REQ-F-016 through REQ-F-021. |
| REQ-F-035 | The client shall submit locally queued creates, updates, and deletes to `POST /api/v1/sync/push` when connected. The server shall acknowledge each accepted change; the client shall retain and retry unacknowledged changes without creating duplicate effects. |
| REQ-F-036 | The client shall retrieve server changes from `GET /api/v1/sync/pull?cursor={cursor}`, apply returned creates, updates, and deletions to its local store, and persist the returned Sync Cursor with the applied changes. Deletions shall be represented so that they are propagated to clients that were offline. |
| REQ-F-037 | The server shall detect an update conflict when a client submits a record version older than the current server version. The client shall preserve both the pending local version and the current server version, show the conflict to the user, and allow the user to select which version to retain; the selected resolution shall be submitted as a subsequent change. |

### 3.3 Non-Functional Requirements

| ID | Requirement |
|---|---|
| REQ-NF-001 | Under normal operating conditions (a local SQLite database pre-populated with 1,000 Entries running on an iOS 15 or later simulator or an Android 10 or later emulator), the system shall save a new Entry and confirm completion within 1 second. |
| REQ-NF-002 | Under normal operating conditions (a local SQLite database pre-populated with 1,000 Entries running on an iOS 15 or later simulator or an Android 10 or later emulator), the system shall return search and filter results within 2 seconds. |
| REQ-NF-003 | The system's core capture, organization, search, and export workflows shall operate without an internet connection. Changes made while offline shall be queued for later synchronization. |
| REQ-NF-004 | The client shall store a durable working copy locally on the device in SQLite using `expo-sqlite`. Persisted data and pending changes shall remain available after the application is closed and reopened, whether or not synchronization has completed. |
| REQ-NF-005 | The system shall preserve valid relationships among Entries, Books, Chapters, Tags, Book Associations, Chapter Associations, and Entry Links when records or associations are created, edited, detached, or deleted. |
| REQ-NF-006 | Deleting a Book or Chapter shall not cause loss of the Content or identity of any associated Entry. |
| REQ-NF-007 | The application shall install and operate on Android 10.0 or later (API 29 or later) and iOS 15.0 or later. |
| REQ-NF-008 | Core workflows shall be usable at a minimum viewport of 360 by 640 dp without clipped interactive controls or unintended horizontal scrolling. |

| ID | Requirement |
|---|---|
| REQ-NF-009 | All interactive UI controls shall have explicit React Native `accessibilityLabel` attributes. The interface shall support platform text scaling without making core workflows inaccessible. |
| REQ-NF-010 | Exported `.txt` files shall use UTF-8 encoding and preserve representative accented and non-Latin text. |
| REQ-NF-011 | If an export cannot be completed, the system shall notify the user, shall not report success, and shall not present a partial file as a completed export. |
| REQ-NF-012 | The system shall maintain an internal, device-local diagnostic/audit log recording the occurrence and time of Entry, Book, and Chapter creation, update, and deletion; association and Entry Link changes; and export operations. Timestamps shall use ISO-8601 format in UTC. The log shall have a maximum size of 2 MB and shall rotate on a first-in, first-out (FIFO) basis by removing the oldest log records when necessary to remain within the limit. |
| REQ-NF-013 | The MVP shall not provide a dedicated user-facing interface for viewing or managing the diagnostic log. |
| REQ-NF-014 | The codebase shall be developed using Test-Driven Development (TDD). Client domain logic and SQLite repositories, plus backend domain logic, API, and synchronization components, shall have automated unit and integration tests with at least 85 percent statement and branch coverage, measured using Jest with `npm test - -coverage` (excluding `*.config.js` and `*.d.ts`). Additionally, the engineering process shall maintain project logs recording requirement conflict resolution, coding-agent (AI co-pilot) interaction prompts and decisions, and key technical trade-offs alongside technical documentation in the project repository. |
| REQ-NF-015 | Client-server API communication shall use HTTPS and UTF-8 encoded JSON, follow the versioned RESTful resource and synchronization interface in this SRS, and use HTTP status codes to distinguish successful requests, invalid requests, and version conflicts. |

# 4 Acceptance Criteria

Each criterion below states observable evidence for its associated requirement. Criteria are concise verification conditions and are not written in GWT form.

## 4.1 Functional Acceptance Criteria

| ID | Acceptance criterion |
|---|---|
| AC-F-001 | A Quick Capture action creates an Entry, and the Entry is available in the Entry list. |
| AC-F-002 | Nonblank Content with no Title can be saved. Blank Content is rejected with a validation message. |
| AC-F-003 | With blank Title, Content of 25 characters or fewer and no newline displays unchanged. When Content exceeds 25 characters, the first 25 body characters are followed by "...". When a newline occurs within the first 25 characters, the body ends before the newline and "..." is appended; no text from subsequent lines appears. The ellipsis does not count toward the 25-character body limit. |
| AC-F-004 | The type selector offers the six specified preset types, and each can be assigned to an Entry. |

| ID | Acceptance criterion |
|---|---|
| AC-F-005 | A General Note can be saved with Content as the only supplied field. |
| AC-F-006 | Viewing an Entry displays its Content, explicit or generated title, Entry Type, Tags, Book and Chapter Associations, and Entry Links. |
| AC-F-007 | Changes to an Entry's Content, Title, Entry Type, Tags, and Book or Chapter Associations persist and appear on subsequent viewing. |
| AC-F-008 | A deleted Entry is absent from lists and search results, and no active Entry Link points to it. |
| AC-F-009 | A Tag can be added to an Entry, appears in its details, and can be removed. |
| AC-F-010 | An unassociated Entry remains available as standalone and can subsequently be associated with a Book or Chapter. |
| AC-F-011 | The user can create, view, edit, and delete Books. Book deletion leaves associated Entries intact. |
| AC-F-012 | The user can create, view, edit, and delete Chapters within a Book. Chapter deletion leaves associated Entries intact. |
| AC-F-013 | A direct Book Association can be created, displayed, and removed. |
| AC-F-014 | One Entry can be associated with at least two Books and appears under each. |
| AC-F-015 | A Chapter Association can be created, displayed, and removed. |
| AC-F-016 | Creating a Chapter Association also creates the parent Book association; the data distinguishes this derived relation from a direct Book Association. |
| AC-F-017 | Deleting a Book or Chapter leaves Entry content intact, removes associations to deleted records, preserves remaining valid associations, and leaves Entries with no associations available as standalone. |
| AC-F-018 | Removing a Chapter Association preserves an independent direct Book Association. A Book association that existed only through the Chapter is removed when that Chapter association is removed and no other relation implies it. |
| AC-F-019 | Two Entries can be linked without a label; linking does not alter their Entry content or type. |
| AC-F-020 | After linking Entries A and B, each Entry displays the other as linked. |
| AC-F-021 | Removing the link from either Entry removes it from both link lists; both Entries remain available. |
| AC-F-022 | A case-insensitive query matching text in either Title or Content returns the Entry; a query with no match returns no such Entry. |
| AC-F-023 | Applying an Entry Type filter returns matching Entries and excludes Entries of other types. |
| AC-F-024 | Applying a Tag filter returns Entries with the selected Tag. When multiple Tags are selected, only Entries with every selected Tag are returned. |
| AC-F-025 | With Search, Type, and Tag active, every result matches all active criteria; an Entry failing any criterion is excluded. When multiple Entry Types or Tags are selected, a result matches every selected value within that category. |
| AC-F-026 | Export creates a local `.txt` file containing the Entry's Content. |
| AC-F-027 | Export creates a local `.txt` file for the selected Book project. |
| AC-F-028 | The export includes directly associated and Chapter-associated Entries; Chapter entries occur beneath their Chapter headings ordered by ascending internal integer position when available, otherwise by ascending `created_at`. Direct Book-level Entries not placed under Chapters occur under "General Book Notes" at the end. |

| ID | Acceptance criterion |
|---|---|
| AC-F-029 | An Entry associated with both a Chapter and its parent Book occurs once, under its Chapter heading, and not again in "General Book Notes". |
| AC-F-030 | API requests create, retrieve, update, and delete an Entry; successful JSON responses include the server version, and deleted Entries are no longer retrievable. |
| AC-F-031 | API requests create, retrieve, update, and delete a Book; the resulting state is reflected by subsequent retrieval. |
| AC-F-032 | API requests create, retrieve, update, and delete a Chapter and preserve its parent-Book relationship. |
| AC-F-033 | API requests create, retrieve, update, and delete a Tag and represent its assignment to an Entry. |
| AC-F-034 | API requests create and delete each supported Entry association and link; tests confirm the Chapter-derived Book relationship, direct-association behavior, symmetric links, and no dangling references after deletion. |
| AC-F-035 | With connectivity restored, each queued local change is acknowledged and applied once; a failed or unacknowledged change remains queued and can be retried without duplicate records or relationships. |
| AC-F-036 | Pulling changes from a saved cursor updates the local store, propagates remote deletions, and advances the cursor only when the changes have been applied and persisted. |
| AC-F-037 | Submitting an outdated record version returns a conflict and exposes both local and server versions in the client; the user can select either version, and the selected version is submitted and synchronized. |

### 4.2 Non-Functional Acceptance Criteria

| ID | Acceptance criterion |
|---|---|
| AC-NF-001 | On an iOS 15+ simulator or Android 10+ emulator with a local SQLite database pre-populated with 1,000 Entries, Entry save and success confirmation complete within 1 second. |
| AC-NF-002 | On an iOS 15+ simulator or Android 10+ emulator with a local SQLite database pre-populated with 1,000 Entries, search and filter results appear within 2 seconds. |
| AC-NF-003 | With network connectivity disabled, capture, organization, search, and export complete using local data; offline changes remain queued after closing and reopening the app. |
| AC-NF-004 | Inspection confirms use of the on-device SQLite database via `expo-sqlite`; saved data and pending changes remain after closing and reopening the app, including before synchronization. |
| AC-NF-005 | Tests of create, edit, associate, detach, and delete operations show no dangling references and show only valid remaining relationships. |
| AC-NF-006 | After Book or Chapter deletion, associated Entry identity and Content remain retrievable. |
| AC-NF-007 | The release application installs and launches on Android API 29+ and iOS 15+. |
| AC-NF-008 | At 360 by 640 dp, core workflows have no clipped interactive controls or unintended horizontal scrolling. |
| AC-NF-009 | Every interactive UI control has an explicit React Native `accessibilityLabel` attribute; platform text scaling does not block completion of core workflows. |

| ID | Acceptance criterion |
|---|---|
| AC-NF-010 | Exported files decode as UTF-8 and preserve representative accented and non-Latin test strings. |
| AC-NF-011 | A simulated or induced export failure produces an error indication, no success indication, and no partial file reported as complete. |
| AC-NF-012 | Inspection confirms that each required event is timestamped in ISO-8601 UTC format and that log rotation retains no more than 2 MB of records by removing the oldest records first. |
| AC-NF-013 | The MVP contains no dedicated user-facing screen or controls for viewing or managing diagnostic logs. |
| AC-NF-014 | Development evidence demonstrates a tests-first workflow with at least 85 percent statement and branch coverage reported by Jest for client domain logic and SQLite repositories, plus backend domain logic, API, and synchronization components. Repository audit also confirms the presence of engineering logs documenting requirement conflict resolution, coding-agent prompts and responses, and architectural decisions. |
| AC-NF-015 | Inspection of client-server traffic confirms HTTPS transport, UTF-8 JSON request and response bodies, versioned REST resource and sync paths, use of standard HTTP methods, and distinct status codes for success, invalid requests, and version conflicts. |

# 5 Behavioral Scenarios (Given–When–Then)

The following scenarios are representative acceptance-level examples. They complement, but do not exhaust, the acceptance criteria above.

### Scenario 1: Generate a Display Title

**Given** an Entry is created with a blank Title
**When** the user saves Content
**Then** the system generates a Display Title whose body is limited to the first 25 characters
**And** if the Content exceeds 25 characters or a newline occurs within the first 25 characters, the system appends "..." after the retained body, outside the 25-character limit
**And** no text after the first newline appears in the Display Title.

### Scenario 2: Create an Entry with All Details

**Given** the user is creating an Entry and has entered an explicit Title and Content, selected an Entry Type, and added Tags
**When** the user saves the Entry
**Then** the Entry is saved with the entered Title and Content, selected Entry Type, and Tags
**And** viewing the Entry shows those saved values without replacing the explicit Title with a generated one.

### Scenario 3: Chapter and Book Associations

**Given** an Entry's only Chapter Association in Book A is to the Chapter being deleted, and it is also directly associated with Book B
**When** the user deletes that Chapter

**Then** the Entry remains available and its association with Book B remains
**And** it is no longer associated with the deleted Chapter or its Chapter-derived Book A association.

**Given** an Entry has no association other than one with a Chapter
**When** the user deletes that Chapter
**Then** the Entry remains available as a Standalone Entry.

**Given** an Entry is associated with a Book
**When** the user deletes that Book
**Then** the Entry remains available and is detached from the deleted Book.

### Scenario 4: Symmetric Entry Links

**Given** Entries A and B exist and are not linked
**When** the user links A to B
**Then** B appears in A's links and A appears in B's links.

**When** the user removes the link from either Entry
**Then** the link no longer appears for either Entry, and both Entries remain available.

### Scenario 5: Search and Combined Filters

**Given** Entries contain matching text in their Titles or Contents using different letter cases
**When** the user searches for that text
**Then** all Entries with a case-insensitive match in either field appear, and Entries without a match do not.

**Given** Entries have different Entry Types and Tags
**When** the user applies a search term, multiple Entry Type values, and multiple Tag values together
**Then** each result matches the search term and every selected Entry Type and Tag
**And** Entries that fail any one of these criteria do not appear.

### Scenario 6: Export Book Project in Chapter Order

**Given** a Book has Chapters in sequential order, Entries associated with those Chapters, and Entries associated directly with the Book
**When** the user exports the Book project
**Then** Chapter-associated Entries appear under their respective Chapter headings, with Chapters sorted by ascending internal position when available and otherwise by ascending `created_at`
**And** Entries associated directly with the Book but not placed under a Chapter appear in a dedicated "General Book Notes" section at the end
**And** an Entry associated with both a Chapter and the Book appears once, under its Chapter heading.

### Scenario 7: Offline Use and Persistence

**Given** the device has no network connection
**When** the user creates and organizes an Entry, searches for it, and exports it
**Then** these operations use local data and complete without network access.

**When** the user closes and reopens the app while still offline
**Then** the Entry and its saved Content remain available and its changes remain queued for synchronization.

**When** network connectivity is restored
**Then** the queued changes are synchronized with the server without requiring the user to recreate the Entry.

### Scenario 8: Create and Structure a General Note

**Given** the user has an idea to capture
**When** the user creates a General Note with Content only
**Then** the Entry is saved as a General Note without requiring a Title, Tag, Book, or Chapter.

**When** the user later changes its Entry Type, adds a Tag, and associates it with a Book or Chapter
**Then** those changes are saved and displayed when the Entry is viewed again.

### Scenario 9: Export Success and Failure

**Given** an Entry contains representative accented and non-Latin text
**When** the user successfully exports the Entry
**Then** a `.txt` file is available in device-local storage and preserves the text when decoded as UTF-8.

**Given** a local storage error prevents an export from completing
**When** the user attempts the export
**Then** the app reports the failure and does not indicate that the export succeeded.

### Scenario 10: Delete an Entry

**Given** an Entry exists and is linked to another Entry
**When** the user deletes the Entry
**Then** the deleted Entry is absent from Entry lists and search results
**And** no active Entry Link points to the deleted Entry
**And** the other linked Entry remains available.

### Scenario 11: Server API Resource Management

**Given** the client is connected to the backend API
**When** the client creates, retrieves, updates, and deletes Entries, Books, Chapters, and Tags using their API resources
**Then** each successful operation returns or reflects the expected JSON resource state
**And** creating or removing associations and Entry Links through their resources preserves the relationship rules and leaves no dangling references.

### Scenario 12: Offline Change Synchronization

**Given** the client has queued local creates, updates, and deletions while offline
**When** connectivity is restored and the client pushes its pending changes and pulls server changes from its saved Sync Cursor
**Then** acknowledged local changes are applied once and remote changes, including deletions, are applied to the local store
**And** unacknowledged changes remain queued for retry
**And** the new Sync Cursor is persisted with the applied changes.

### Scenario 13: Resolve a Synchronization Conflict

**Given** a record was changed both locally and on the server since the client's last synchronization
**When** the client submits its outdated version
**Then** the server reports a version conflict and the client preserves and displays both versions
**When** the user selects which version to retain
**Then** the selected version is submitted as a new change and synchronized.

# 6 Traceability and Outstanding Specification Details

The acceptance criteria are keyed directly to requirement identifiers. Functional criteria `AC-F-001` through `AC-F-037` map one-to-one to `REQ-F-001` through `REQ-F-037`; non-functional criteria `AC-NF-001` through `AC-NF-015` map one-to-one to `REQ-NF-001` through `REQ-NF-015`. The GWT scenarios exercise key cross-requirement workflows and cite the same observable behavior. The accompanying RTM maps each requirement to its acceptance criterion and relevant scenario or verification evidence.

The following details were not decided during SKED and must be specified in the design, verification plan, or a later SRS revision before their related checks can be fully reproducible:

- whether users can manually reorder Chapters, beyond the export ordering rule specified in REQ-F-028;
- exact text layout, metadata, file naming, and platform destination-selection behavior for exports;
- diagnostic-log redaction and timestamp precision;
- account identity, authentication, and authorization behavior, which are outside this revision's specified scope; and
- the precise definition of domain logic and repository code included in the REQ-NF-014 coverage scope.